

POS Kiosk — Technical Manual

Architecture, environment setup, build, packaging and troubleshooting for the Flutter Windows kiosk app.

Written for

Developers, Integrators & IT Staff

Contents

1. Overview & architecture
2. Prerequisites
3. Environment configuration
4. Install, generate, run
5. Code generation
6. Navigation & routes
7. Feature → backend endpoint map
8. Windows specifics
9. Building a release & packaging
10. Troubleshooting

Section numbers above match the numbered banners in the body. Use your PDF reader's bookmarks / outline panel to jump between sections.

NOTE Commands are shown for a Windows developer machine using PowerShell or Git Bash. The kiosk app lives in the `kiosk/` folder of the repository.

1. Overview & architecture

The kiosk is a Flutter desktop application targeting Windows (Android is also buildable). It is a thin client: all business logic and persistence live in the NestJS backend.

1.1 Stack

Concern	Choice
UI toolkit	Flutter (Dart <code>^3.7.2</code>), pinned via FVM (<code>.fvmrc</code>)
State management	Hooks Riverpod (<code>hooks_riverpod</code> + <code>flutter_hooks</code>)
Navigation	GoRouter with typed routes (<code>go_router_builder</code> codegen)
Serialization	<code>dart_mappable</code> (<code>*.mapper.dart</code>)
HTTP	Dio, with an auth interceptor
Config	<code>envied</code> (compile-time env, <code>env.g.dart</code>)
Windows integration	<code>window_manager</code> , <code>win32</code> , <code>screen_retriever</code> , <code>desktop_multi_window</code>
Printing	<code>esc_pos_utils_plus</code> (ESC/POS receipts)
Assets	<code>flutter_gen</code> (<code>lib/gen/assets.gen.dart</code>)

1.2 Data flow

```
View (HookConsumerWidget)
  -> Riverpod provider (state/)
    -> Repository (repositories/)
      -> API source (lib/data/backend_api/sources/)
        -> Dio client (api_clients.dart, auth interceptor)
          -> Backend http://<host>:3000/api/v1
```

1.3 Feature module layout

```
lib/features/<feature>/
  entities/      immutable domain models + .mapper.dart
  repositories/  abstract interface + Impl calling the API layer
  state/         Riverpod AsyncNotifier / StateNotifier providers
  use_cases/     (optional) single-responsibility operations
  view/          screens & dialogs (HookConsumerWidget)
```

1.4 HTTP layer

```
lib/data/backend_api/  
  sources/      one class per backend resource (ProductsApi, ...)  
  schemas/      @MappableClass DTOs mirroring backend DTOs  
  mappers/      custom dart_mappable type mappers  
                  (ImageUrlMapper, DateTimeWithOffsetMapper, ...)  
  api_clients.dart Dio provider wired with the auth interceptor
```

NOTE Clean-architecture rule: `lib/data/` must never import `lib/features/`. Error translation (`DioException` → `ApiException`) belongs in the data layer.

2. Prerequisites

Tool	Version	Notes
Flutter SDK	3.x (stable), pinned by FVM	<code>dart pub global activate fvm</code> then <code>fvm use stable</code>
Dart SDK	<code>^3.7.2</code>	Bundled with Flutter
Node.js	18+	Only needed if you also build the backend / installer
Visual Studio (Desktop C++)	2022	Required by <code>flutter build windows</code>
A running backend	—	The app checks <code>/api/v1/health/live</code> on start-up

Confirm the toolchain with `fvm flutter doctor`.

3. Environment configuration

Copy `kiosk/.env.sample` to `kiosk/.env` and fill every value. Variables are read at **compile time** by `envied`, so you must re-run code generation after changing them.

Variable	Purpose
<code>BACKEND_API_BASE_URL</code>	Base URL of the NestJS API, including <code>/api/v1</code> (e.g. <code>http://localhost:3000/api/v1</code>).
<code>SECURE_STORAGE_KEY</code>	Key used to encrypt tokens in secure local storage.
<code>ORDERS_LIVE_FEED_WS_URL</code>	WebSocket URL for the live channel-orders feed (Orders board).
<code>CARTIVO_AUTH_API_BASE_URL</code>	Base URL for the external ordering-platform auth service.
<code>ORDERS_EVENTS_API_BASE_URL</code>	Base URL for the channel-order events / history API.
<code>CLIENT_ID</code>	Client identifier for the external orders integration.
<code>WEBHOOK_SECRET</code>	Shared secret used to verify inbound order webhooks.

NOTE Every new variable must also be declared as an `@EnviedField()` on the `Env` class in `lib/config/environment/`, or the build will not see it.

4. Install, generate, run

4.1 First-time setup

```
cd kiosk
dart pub global activate fvm          # once per machine
fvm use stable                       # install the pinned Flutter
cp .env.sample .env                 # then edit .env
fvm flutter pub get
fvm dart run build_runner build --delete-conflicting-outputs
```

4.2 Run

```
fvm flutter run -d windows           # desktop
fvm flutter run -d android          # android (optional)
```

In dev mode `main.dart` sizes the window to **1536×864**; in a release build it goes full-screen.

4.3 Analyze

```
fvm dart analyze
```

NOTE For **MOBILE-ONLY** work in the sibling `mobile/` and `mobile_merchant/` apps, verify with `dart analyze` and do not run or modify `be/` or `kiosk/`.

5. Code generation

Four generators run through `build_runner`. Re-run it after touching any annotated file. Never hand-edit generated output.

Generator	Produces	From
<code>dart_mappable</code>	<code>*.mapper.dart</code>	<code>@MappableClass</code> types (entities, API schemas)
<code>envied_generator</code>	<code>env.g.dart</code>	The <code>Env</code> class + <code>.env</code>
<code>go_router_builder</code>	<code>router.g.dart</code>	<code>@TypedGoRoute</code> route classes
<code>flutter_gen</code>	<code>lib/gen/assets.gen.dart</code>	<code>pubspec.yaml</code> assets

```
fvm dart run build_runner build --delete-conflicting-outputs
# or, while iterating:
fvm dart run build_runner watch --delete-conflicting-outputs
```


6. Navigation & routes

Routes are defined as typed `GoRouteData` classes under `lib/navigation/` and assembled in `router.dart`. Initial location is `StartupRoute`.

Route	Path	Screen
StartupRoute	/	Backend health poll, then redirects to Login
OnboardingRoute	/onboarding	Welcome / Touch To Start
LoginRoute	/login	Roster + PIN pad
SetupPinRoute	/setup-pin	Forced PIN change
MenuRoute	/menu	Main menu grid (role-filtered)
OrderingRoute	/ordering	New Order (product grid + order panel)
CartRoute / DiscountRoute / PaymentRoute / ReceiptRoute	/cart ...	Checkout flow
OrdersRoute	/orders	Channel-orders kanban board
TransactionsRoute	/transactions	Sales history + report launchers
RefundRoute	/refund	Process Refund
ProductsRoute (Inventory)	/inventory	Inventory Management (Products / Categories)
UserManagementRoute	/user-management	Employee accounts
DeviceTransferRoute	/device-transfer	Backup & Transfer hub
CashierReportRoute / CashierDailyReportRoute / ZReadingRoute / CashierReportsRoute	/cashier-report , /z-reading , ...	X-Reading, Cashier Daily Report, Z-Reading, history (with /history/:id detail variants)

NOTE A `WindowsTouchKeyboardObserver` is registered as a navigator observer to manage the on-screen keyboard.

7. Feature → backend endpoint map

Which API each screen talks to (all under `/api/v1`). Keep DTO schemas in `lib/data/backend_api/schemas/` in sync with the backend.

Feature / screen	Primary endpoints
Start-up	<code>GET /health/live</code>
Login roster / PIN	<code>GET /users/roster</code> , <code>POST /auth/login/pin</code> , <code>POST /auth/refresh</code> , <code>GET /auth/me</code>
PIN change	<code>PATCH /users/:id</code> , <code>POST /users/verify-pin</code>
Main menu / terminal gate	<code>GET /pos-terminals/my-terminal</code>
New Order (products)	<code>GET /catalog/products</code> , <code>GET /catalog/categories</code> , <code>GET /products/:id</code> , <code>GET /product-variants/by-product/:id</code> , <code>GET /catalog/modifier-groups</code>
Checkout / sale	<code>POST /sales-orders</code> , <code>PATCH /sales-orders/:id/item/:itemId</code> , <code>PATCH /sales-orders/:id/discount</code> , <code>PATCH /sales-orders/:id/confirm</code> , <code>POST /payments</code>
Transactions	<code>GET /sales-orders</code> , <code>GET /sales-orders/:id/with-items</code>
Void / Refund	<code>PATCH /sales-orders/:id/void</code> , <code>POST /refunds</code> , <code>POST /users/verify-pin</code>
Orders board	<code>WS ORDERS_LIVE_FEED_WS_URL</code> , <code>ORDERS_EVENTS_API_BASE_URL</code> , <code>CARTIVO_AUTH_API_BASE_URL</code>
Inventory Management	<code>GET/POST/PATCH /products</code> , <code>POST /products/import-csv</code> , <code>catalog/admin category endpoints</code> , <code>/product-variants</code> , <code>/modifier-groups</code>
User management	<code>GET/POST /users</code> , <code>PATCH/DELETE /users/:id</code> , <code>GET /users/authorizers</code>
Settings / terminal	<code>GET/PATCH /pos-terminals/my-terminal</code> , <code>POST /pos-terminals/register</code> , <code>.../my-terminal/payment-methods</code>
X / Daily / Z readings	<code>reports/cashier-x-reading*</code> , <code>reports/cashier-daily-report*</code> , <code>reports/z-reading*</code>
Backup & Transfer	<code>POST /device-transfer/export</code> , <code>POST /device-transfer/import</code>

8. Windows specifics

8.1 Fullscreen & DPI

- `main.dart` initialises `window_manager` ; production builds go borderless full-screen, dev builds use 1536×864.
- `win32` is used for high-DPI awareness.

8.2 Customer display second window

`main.dart` checks the window argument: the normal engine runs the cashier app, a second engine (`desktop_multi_window`) runs the customer display. `screen_retriever` finds the non-primary monitor (falling back to visible-position comparison when a display reports an empty id) and positions the window there without stealing focus. `CustomerDisplayHost` in the cashier engine pushes debounced snapshots to it.

8.3 On-screen keyboard

Text fields use `KeyboardSuppress` helpers plus a `PhysicalKeyboardDetector` so that a hardware keyboard, when attached, disables the touch keyboard automatically.

8.4 Printing

Receipts and X/Z/daily reports are encoded to ESC/POS byte streams (`encode_esc_pos_*` use cases) and also rendered to PDF (`render_*_pdf`) for preview / archive.

9. Building a release & packaging

9.1 Standalone Windows build

```
cd kiosk
fvm flutter build windows --release
# output: kiosk\build\windows\x64\runner\Release\
#   pos_app.exe, *.dll, data\
```

9.2 As part of the one-click installer

From the repo root, `build-installer.bat` runs Flutter codegen, then builds the Flutter app and the backend SEA in parallel, then compiles the Inno Setup installer. See the *Backend — Technical Manual* for the full installer walkthrough.

Installed component	Location	Runs as
Flutter app (<code>pos_app.exe</code>)	<code>C:\POSKiosk\</code>	Desktop app / autostart
Backend SEA (<code>POSBackend.exe</code>)	<code>C:\POSKiosk\backend\</code>	Service <code>POSBackendService</code> (NSSM)
PostgreSQL 16	<code>C:\POSKiosk\pgsql\</code>	Service <code>POSPostgres</code>

NOTE Runtime kiosk identity on an installed machine comes from `C:\POSKiosk\settings.txt` (`kiosk.no=<n>`), written by the installer wizard — not from `.env`.

10. Troubleshooting

Symptom	Cause / fix
App stays on start-up screen	Health poll to <code>/health/live</code> failing. Check <code>POSBackendService</code> and <code>POSTgres</code> are Running; check <code>BACKEND_API_BASE_URL</code> . On an installed machine run <code>C:\POSKiosk\scripts\recover-services.bat</code> as admin.
<code>build_runner</code> errors after a pull	Run with <code>--delete-conflicting-outputs</code> ; if still broken, <code>fvm flutter clean && fvm flutter pub get</code> then regenerate.
Env value not picked up	It was added to <code>.env</code> but not to the <code>Env</code> class, or codegen wasn't re-run.
Blank product grid vs. real error	The grid distinguishes empty-category from load failure; a failure shows "Could not load products" with Retry — inspect the Dio error / token.
Customer display on wrong monitor	Empty display ids on some hardware; fix the Windows primary display or cabling and restart.
"Internal server error" creating a sale	Backend can't resolve the kiosk number — verify <code>settings.txt</code> and restart the backend service.

Log locations (installed machine)

Log	Contents
<code>C:\POSKiosk\logs\backend-output.log</code> / <code>backend-error.log</code>	Live NestJS stdout / stderr
<code>C:\POSKiosk\logs\setup-postgres-install.log</code>	DB init, service registration, migrations
<code>C:\posdata\log\postgresql.log</code>	PostgreSQL server log
<code>customer-display-receiver.log</code>	Customer-display sub-window diagnostics